Fundação Escola de Comércio Álvares Penteado

Ciência da Computação 4º Semestre

Sistemas Embarcados e Robótica

Eduardo Chen Zou – 24026559
Lucas Kenichi Soares – 24026179
Pedro Della Rosa Antônio – 24026018
Pedro Dimitry Zyrianoff – 24026165

São Paulo – 2026

Entrega 02 — Programação e Controle de Dispositivos no Arduino

Projeto: Smart Tênis — Alvo a Laser para Treino Autônomo

1. Objetivo do projeto

O Smart Tênis é um sistema de treino de tênis que transforma a prática em um jogo. A ideia do nosso grupo é simples: um ponto de laser aparece em uma superfície branca (como um quadro ou um lençol translúcido), que funciona como alvo. O jogador joga a bola tentando acertar esse ponto, e o sistema verifica automaticamente se ele acertou, passou perto ou errou. Como o ponto de laser se move, o treino fica mais dinâmico e mais desafiador, sem precisar de outra pessoa para posicionar o alvo.

Para isso, o projeto é dividido em quatro partes que trabalham juntas:

- **Aplicativo Android:** o jogador entra com sua conta, escolhe o nível de dificuldade (fácil, médio ou difícil) e inicia a partida. O app também mostra a pontuação.
- **Servidor Flask (no notebook):** recebe as ordens do aplicativo pela rede e as repassa ao Arduino pelo cabo USB. Ele também calcula para onde o laser deve apontar a cada instante.
- **Arduino Uno:** funciona como o "braço" do sistema: recebe os comandos, move os dois servos que apontam o laser, liga o laser e dá avisos por LEDs e buzzer. Esta é a parte apresentada nesta entrega.
- **Câmeras e visão computacional:** duas câmeras observam a bola e, com inteligência artificial, identificam onde ela caiu em relação ao alvo. Quando há acerto, o sistema avisa o servidor e o aplicativo soma os pontos. Essa parte ainda está em desenvolvimento e treinamento.

Cada nível muda o comportamento do laser: no fácil, ele fica parado no centro; no médio, desenha um círculo lento; no difícil, muda para um ponto aleatório a cada 2 segundos. Por segurança, cada partida termina sozinha após 70 segundos, para que o laser não fique ligado sem controle.

Em resumo, o funcionamento é: o jogador inicia a partida no aplicativo → o notebook envia os comandos ao Arduino → o Arduino aponta o laser → o jogador lança a bola → as câmeras conferem o acerto → o aplicativo mostra a pontuação.

Nesta entrega, o foco é o Arduino, a parte que obedece aos comandos e controla os dispositivos físicos. Nas próximas seções mostramos como ele foi montado, o código-fonte e como testar o funcionamento.

2. Objetivo do programa

Esta entrega apresenta o firmware (hoje_com_G.ino) do Arduino Uno que controla o sistema de mira a laser do Smart Tênis. O Arduino recebe comandos de um caractere pela porta serial e controla atuadores em tempo real: dois servos SG90 (eixos X e Y) que posicionam o laser, o próprio laser, dois LEDs e um buzzer. Por isso ele pode ser testado direto pelo Serial Monitor, mesmo sem o restante do sistema.

Além do ajuste em passos de 5° (comandos A, D, W e S), o firmware aceita o posicionamento absoluto G<x>,<y>, usado pelo laser automático para atualizar os servos até 20 vezes por segundo, e responde pela serial com o estado de cada ação, o que permite acompanhar o funcionamento no Serial Monitor.

3. Componentes utilizados

Componente	Tipo	Função no circuito
Arduino Uno	Controlador	Executa o programa, lê os comandos recebidos pela serial e comanda os atuadores
Cabo USB / porta serial	Entrada (serial)	Recebe os comandos de um caractere (A, D, W, S, L, E, B, R, G, P, ?) a 9600 baud e devolve as respostas ao computador
Micro servos SG90 (Servo1 e Servo2)	Atuador	Movem o laser nos eixos X (Servo1) e Y (Servo2), de 0° a 180°
Laser	Atuador (digital)	Emite o ponto de luz do alvo; liga e desliga pelo comando L
LED1 e LED2 + resistores	Atuador (digital)	Indicadores luminosos, alternados pelo comando E
Buzzer	Atuador (sonoro)	Bipe de 1000 Hz por 200 ms (comando B)
Protoboard e jumpers	Montagem	Ligações entre os componentes e o Arduino

4. Ligações no Arduino

Pino	Componente	Tipo de porta	Observação
D2	Servo1 (fio de sinal)	Saída PWM (Servo)	Eixo X; fio vermelho em 5V e marrom em GND
D3	Servo2 (fio de sinal)	Saída PWM (Servo)	Eixo Y; fio vermelho em 5V e marrom em GND
D8	LED2	Saída digital	Em série com 330 Ω; catodo no GND
D9	LED1	Saída digital	Em série com 330 Ω; catodo no GND
D10	Buzzer	Saída digital (tone)	Outro terminal no GND
D11	Laser	Saída digital	Liga/desliga pelo comando L; outro terminal no GND
USB (D0/D1)	Porta serial	Comunicação serial	Cabo USB ao computador, a 9600 baud: entram os comandos e saem as respostas

5. Código-fonte comentado

O código foi escrito na Arduino IDE e está completo abaixo, sem alterações em relação ao firmware hoje_com_G.ino. Também acompanha esta entrega o arquivo hoje_com_G.ino.

```
// ============================================================================
// hoje_com_G.ino  -  copia do hoje.ino + posicionamento ABSOLUTO dos servos
//
// Novos comandos (o resto e identico ao hoje.ino):
//   G<x>,<y>\n   move servo1 para x e servo2 para y (0-180). SILENCIOSO: nao responde,
//                para o Flask poder mandar 20x/s sem lotar a serial.  Ex.: "G90,120\n"
//   P            responde "[POS] <servo1>,<servo2>". O Flask usa isto para descobrir
//                se o firmware aceita G (o sketch antigo ignora o P).
//
// Linha malformada em G (ex.: "G\n" ou "Gabc") e IGNORADA: nao move para 0,0.
// Para gravar: feche o Serial Monitor e o Flask (eles seguram a COM), abra este arquivo
// no Arduino IDE e clique em Upload.
// ============================================================================
#include <Servo.h>

// Servos
Servo servo1;
Servo servo2;

// Pinos
const int LED1 = 9;
const int LED2 = 8;
const int BUZZER = 10;
const int SERVO1_PIN = 2;
const int SERVO2_PIN = 3;
const int LASER_PIN = 11;

// Variáveis
int angle1 = 90;
int angle2 = 90;
bool laser_on = false;

void setup() {
  // Inicializa pinos
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  pinMode(LASER_PIN, OUTPUT);

  // Inicializa servos
  servo1.attach(SERVO1_PIN);
  servo2.attach(SERVO2_PIN);

  // Posição inicial (centro)
  servo1.write(90);
  servo2.write(90);
```

```
  // Laser desligado inicialmente
  digitalWrite(LASER_PIN, LOW);

  Serial.begin(9600);
  Serial.setTimeout(30);   // G: le a linha em ate 30 ms (nao trava o loop)

  // Menu
  Serial.println("========== SISTEMA PRONTO ==========");
  Serial.println("SERVOS:");
  Serial.println("  A/D = Servo1 esquerda/direita");
  Serial.println("  W/S = Servo2 cima/baixo");
  Serial.println();
  Serial.println("LASER:");
  Serial.println("  L = Laser liga/desliga");
  Serial.println();
  Serial.println("OUTROS:");
  Serial.println("  E = Alterna LED 1 e LED 2");
  Serial.println("  B = Buzzer faz BIP");
  Serial.println("  R = Reset servos (90°)");
  Serial.println("  G<x>,<y> = Servos em posicao absoluta");
  Serial.println("  P = Mostra posicao dos servos");
  Serial.println("  ? = Mostra este menu");
  Serial.println("====================================");
  Serial.println();
}

void loop() {
  if (Serial.available()) {
    char cmd = Serial.read();

    // Remove quebras de linha
    if (cmd == '\n' || cmd == '\r') return;

    // ========== SERVO 1 ==========
    if (cmd == 'A' || cmd == 'a') {
      angle1 = constrain(angle1 + 5, 0, 180);
      servo1.write(angle1);
      Serial.print("[SERVO1] Ângulo: ");
      Serial.print(angle1);
      Serial.println("°");
    }

    if (cmd == 'D' || cmd == 'd') {
      angle1 = constrain(angle1 - 5, 0, 180);
      servo1.write(angle1);
      Serial.print("[SERVO1] Ângulo: ");
      Serial.print(angle1);
      Serial.println("°");
    }

    // ========== SERVO 2 ==========
    if (cmd == 'W' || cmd == 'w') {
      angle2 = constrain(angle2 + 5, 0, 180);
      servo2.write(angle2);
      Serial.print("[SERVO2] Ângulo: ");
      Serial.print(angle2);
      Serial.println("°");
    }

    if (cmd == 'S' || cmd == 's') {
      angle2 = constrain(angle2 - 5, 0, 180);
      servo2.write(angle2);
      Serial.print("[SERVO2] Ângulo: ");
      Serial.print(angle2);
      Serial.println("°");
    }

    // ========== LASER (Pino 11) ==========
    if (cmd == 'L' || cmd == 'l') {
      laser_on = !laser_on;
      digitalWrite(LASER_PIN, laser_on ? HIGH : LOW);
      Serial.print("[LASER] ");
```

```
      Serial.println(laser_on ? "LIGADO ✓" : "DESLIGADO ✗");
    }

    // ========== LEDs ==========
    if (cmd == 'E' || cmd == 'e') {
      digitalWrite(LED1, !digitalRead(LED1));
      digitalWrite(LED2, !digitalRead(LED2));
      Serial.println("[LEDs] Alternados");
    }

    // ========== BUZZER ==========
    if (cmd == 'B' || cmd == 'b') {
      tone(BUZZER, 1000);  // Frequência 1000Hz
      delay(200);
      noTone(BUZZER);
      Serial.println("[BUZZER] BIP!");
    }

    // ========== RESET ==========
    if (cmd == 'R' || cmd == 'r') {
      angle1 = 90;
      angle2 = 90;
      servo1.write(90);
      servo2.write(90);
      Serial.println("[RESET] Servos em 90°");
    }

    // ========== POSICAO ABSOLUTA (novo) ==========
    if (cmd == 'G' || cmd == 'g') {
      String linha = Serial.readStringUntil('\n');
      int x, y;
      if (sscanf(linha.c_str(), "%d,%d", &x, &y) == 2) {
        angle1 = constrain(x, 0, 180);
        angle2 = constrain(y, 0, 180);
        servo1.write(angle1);
        servo2.write(angle2);
      }
    }

    if (cmd == 'P' || cmd == 'p') {
      Serial.print("[POS] ");
      Serial.print(angle1);
      Serial.print(",");
      Serial.println(angle2);
    }

    // ========== MENU ==========
    if (cmd == '?') {
      Serial.println();
      Serial.println("COMANDOS:");
      Serial.println("  A/D = Servo1   |  W/S = Servo2");
      Serial.println("  L = Laser      |  E = LEDs");
      Serial.println("  B = Buzzer     |  R = Reset");
      Serial.println("  ? = Menu");
      Serial.println();
    }
  }
}
```

6. Descrição do funcionamento e da lógica

6.1 Fluxo geral

O programa segue o mesmo padrão entrada → processamento → saída definido na arquitetura do Smart Tênis:

- **setup():** executado uma vez ao ligar. Configura os pinos de saída (LEDs, buzzer e laser), associa os servos aos pinos 2 e 3, leva os dois servos à posição central (90°), mantém o laser desligado, inicia a serial a 9600 baud com timeout de 30 ms e imprime o menu de comandos. Corresponde ao estado INICIALIZANDO da máquina de estados do projeto.
- **loop():** repetido continuamente (estado AGUARDANDO COMANDO). A cada ciclo: (1) se há um byte na serial, lê um caractere (quebras de linha são ignoradas); (2) compara com os comandos conhecidos; (3)

executa a ação (move servo, alterna laser ou LEDs, toca o buzzer, reseta ou informa a posição) e envia a resposta pela serial.

6.2 Lógica de cada etapa

- **Leitura de comandos:** Serial.available() verifica se chegou algum byte e Serial.read() lê um caractere por ciclo. As quebras de linha (\n e \r) são descartadas com return, e letras maiúsculas e minúsculas são aceitas (A ou a, D ou d, e assim por diante).
- **Servos em passos de 5°:** A e D mudam angle1 (Servo1), e W e S mudam angle2 (Servo2), em ±5°. A função constrain(..., 0, 180) mantém o ângulo dentro do curso do servo, servo.write() envia o novo ângulo e a resposta [SERVO1] ou [SERVO2] mostra o valor na serial.
- **Posição absoluta G<x>,<y>:** ao receber G, o programa lê o resto da linha com readStringUntil('\n') (no máximo 30 ms) e extrai x e y com sscanf. Se os dois números forem lidos, os ângulos são limitados a 0°–180° e os servos vão direto para a posição. Linha malformada é ignorada e não move os servos. O comando é silencioso (não responde), para poder ser enviado 20 vezes por segundo sem lotar a serial.
- **Laser e LEDs (condicionais):** L inverte a variável laser_on e escreve HIGH ou LOW no pino 11 com digitalWrite, usando o operador ternário. E lê o estado de LED1 e LED2 com digitalRead e grava o oposto com digitalWrite, alternando os dois.
- **Buzzer:** B gera um tom de 1000 Hz com tone(), espera 200 ms com delay() e silencia com noTone(), respondendo [BUZZER] BIP!.
- **Reset, posição e menu:** R volta os dois servos a 90°. P responde [POS] <servo1>,<servo2>; o Flask usa essa resposta para descobrir se o firmware aceita o comando G. ? imprime o menu de comandos.

6.3 Comportamento esperado (valores calculados)

Exemplo de sequência de comandos a partir do estado inicial (servos em 90° e laser desligado). Os valores são calculados a partir da lógica do código e podem ser conferidos no Serial Monitor a 9600 baud, com a opção "Nova linha" para o comando G.

Comando enviado	Servo1 / Servo2 antes	Ação executada	Servo1 / Servo2 depois	Resposta na serial
A	90 / 90	Servo1 +5°	95 / 90	[SERVO1] Ângulo: 95°
W	95 / 90	Servo2 +5°	95 / 95	[SERVO2] Ângulo: 95°
G30,150	95 / 95	Posição absoluta	30 / 150	(nenhuma: silencioso)
P	30 / 150	Consulta a posição	30 / 150	[POS] 30,150
L	30 / 150	Laser desligado → ligado	30 / 150	[LASER] LIGADO ✓
R	30 / 150	Reset dos servos	90 / 90	[RESET] Servos em 90°

6.4 Conteúdos da disciplina aplicados

Conteúdo	Onde aparece no código
Variáveis e constantes	const int para os pinos; variáveis angle1, angle2 e laser_on (bool); objetos servo1 e servo2; cmd (char) e linha (String)
Funções e bibliotecas	biblioteca Servo (attach e write); constrain(), sscanf() e readStringUntil() para limitar e interpretar os valores
Portas digitais	pinMode(...) configura os pinos de saída; digitalWrite(...) liga e desliga o laser e alterna os LEDs; digitalRead(...) lê o estado dos LEDs para invertê-los
PWM e tom	sinal PWM dos servos gerado pela biblioteca Servo (servo1.write e servo2.write); tone() gera o sinal de 1000 Hz do buzzer
Estruturas condicionais	sequência de if comparando o caractere recebido (A, D, W, S, L, E, B, R, G, P, ?); operador ternário no laser; if com sscanf para validar o comando G
Estruturas de repetição	loop() executado continuamente, verificando a serial a cada ciclo

Conteúdo	Onde aparece no código
Comunicação serial	Serial.begin(9600), Serial.setTimeout(30), Serial.read() para os comandos e Serial.print/println para as respostas e o menu

7. Evidência prática

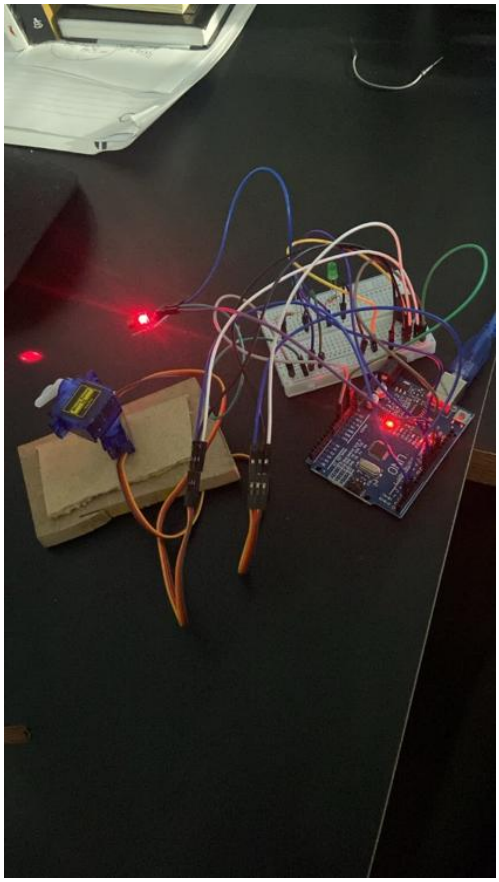


Figura 1 — Circuito montado e energizado: Arduino Uno, protoboard, servo SG90, laser e LEDs ligados por jumpers.

O que a evidência demonstra: o circuito da seção 4 montado e energizado, com a porta serial como entrada e os servos, o laser, os LEDs e o buzzer como atuadores, todos controlados pelo programa da seção 5. A foto mostra o Arduino Uno alimentado pelo cabo USB, a protoboard com os jumpers, o servo SG90 fixado sobre uma base de papelão, o LED verde na protoboard e o emissor vermelho aceso, com o ponto de luz vermelha projetado sobre a mesa.

Como o sistema funciona (passo a passo):

- **Energização e inicialização:** ao conectar o cabo USB, o Arduino Uno liga o circuito e executa o setup(): os servos vão para 90°, o laser fica desligado e o menu de comandos aparece no Serial Monitor. Em seguida o sistema fica no loop(), aguardando comandos.
- **Entrada (serial):** cada caractere enviado pelo Serial Monitor (ou, no sistema completo, pelo Flask) chega pela USB a 9600 baud. O loop() lê o caractere, identifica o comando e executa a ação correspondente, respondendo pela serial.
- **Servos SG90 (D2 e D3):** A e D movem o Servo1 e W e S movem o Servo2 em passos de 5°, limitados a 0°–180°. O comando G<x>,<y> posiciona os dois de uma só vez. Na foto, um servo aparece apoiado sobre a base de papelão, à esquerda.
- **Laser (D11):** o comando L liga e desliga o laser. Quando está ligado, ele projeta o ponto de luz vermelha visível sobre a mesa na foto.
- **LEDs (D8 e D9):** o comando E alterna o estado de LED1 e LED2, servindo de indicador luminoso. O LED verde aparece na protoboard.
- **Buzzer (D10):** o comando B emite um bipe de 1000 Hz por 200 ms, confirmando o comando também de forma sonora.

- **Serial Monitor:** cada ação responde com uma linha de status, como [SERVO1] Ângulo: 95° ou [LASER] LIGADO. O comando P mostra a posição atual dos servos e R devolve os dois a 90°.